

QUIC Traffic Analysis: Stream Identification, Header Analysis, Pattern Recognition & Policing Strategy

Prepared by: Suvidh Jain

Date: 9/3/26

1. Types of Streams That QUIC Traffic Carries

1.1 QUIC Stream

A QUIC stream is a lightweight, ordered, byte-stream abstraction within a single QUIC connection. Unlike TCP, where you get exactly one byte stream per connection, QUIC multiplexes many streams inside a single connection. This is a important architectural difference.

Each stream is identified by a 62-bit Stream ID. The two least significant bits of this ID encode two pieces of information:

- **Bit 0 (LSB):** Identifies the initiator. If 0, the stream was initiated by the client. If 1, the server initiated it.
- **Bit 1:** Identifies directionality. If 0, the stream is bidirectional. If 1, unidirectional.

This gives us four stream types:

Stream Type	Initiator	Directionality	Stream ID Pattern
0x0	Client	Bidirectional	0, 4, 8, 12, ...
0x1	Server	Bidirectional	1, 5, 9, 13, ...
0x2	Client	Unidirectional	2, 6, 10, 14, ...
0x3	Server	Unidirectional	3, 7, 11, 15, ...

This is actually observable if we can parse the STREAM frames, but post-handshake, all frames are encrypted. The *number* and *pattern* of streams being opened influences the observable packet-level behavior.

1.2 HTTP/3 Stream Mapping

HTTP/3 is by far the most common application protocol running over QUIC today (and the primary reason QUIC exists in its current form).

HTTP/3 defines several stream types:

Request Streams (Bidirectional, Client-initiated): Every HTTP request-response pair gets its own bidirectional stream. The client sends the request headers (and optionally body) on this stream, and the server sends the response headers and body back on the same stream. Stream IDs 0, 4, 8, 12, and so on. This is where the bulk of the data transfer happens.

Control Streams (Unidirectional): Both client and server open exactly one control stream each. These carry HTTP/3-specific control frames like SETTINGS, GOAWAY, MAX_PUSH_ID, etc. They are long-lived. They stay open for the entire duration of the connection. The data volume on these streams is minimal.

QPACK Encoder/Decoder Streams (Unidirectional): HTTP/3 uses QPACK for header compression (analogous to HPACK in HTTP/2). Both endpoints open two unidirectional streams for QPACK - one encoder stream and one decoder stream. Again, these are low-volume, long-lived streams.

Push Streams (Unidirectional, Server-initiated): If server push is used (rare, most deployments have disabled it), the server opens unidirectional streams to push resources to the client.

So in a typical HTTP/3 session, you will see:

- 2 control streams (one per side)
- 4 QPACK streams (encoder + decoder, per side)
- N request streams, where N is the number of HTTP requests

This means even a simple web page load that triggers, say, 50 HTTP requests will create 50 bidirectional streams plus the 6 overhead streams. All of these are multiplexed inside a *single* QUIC connection, which is a *single* UDP 4-tuple flow. This multiplexing has consequences for traffic analysis, we cannot separate individual resource fetches by looking at flow-level data alone.

1.3 Non-HTTP/3 Protocols Over QUIC

While HTTP/3 dominates QUIC usage today, the protocol was designed to be general-purpose. Other protocols that use or are being developed to use QUIC include:

DNS over QUIC (DoQ, RFC 9250): DNS queries and responses carried over QUIC. Each query-response pair typically uses its own stream. These are extremely short-lived i.e. a handful of small packets and then the stream is done.

WebTransport: An API that allows web applications to establish bidirectional and unidirectional streams directly over QUIC (or HTTP/3 tunnels). Used for real-time applications like gaming, live collaboration, and media streaming.

SMB over QUIC: Microsoft has implemented SMB (file sharing) over QUIC for remote file access scenarios. This generates traffic that looks quite different from web browsing.

MASQUE: Proxying protocols using QUIC, including IP and UDP proxying. Used in VPN-like scenarios and Apple's iCloud Private Relay.

The Application-Layer Protocol Negotiation (ALPN) extension in the TLS handshake within the Initial packets identifies which application protocol is being used. Common ALPN values include "h3" for HTTP/3, "doq" for DNS over QUIC, etc.

1.4 Stream Lifecycle

A QUIC stream goes through the following states:

Idle → Open → Half-Closed (one direction) → Closed

Streams can also be reset (abruptly terminated) using RESET_STREAM frames, and a receiver can signal that it no longer wants data using STOP_SENDING frames.

For our project, the important observations are:

- **Stream creation and teardown generates control overhead.** Each new stream requires at least one STREAM frame, and flow control updates may follow. When many streams are opened in quick succession (like a web page loading many resources), there is a measurable burst of activity.
- **Streams have flow control at two levels:** per-stream and connection-level. The receiver advertises how much data it is willing to accept. This creates natural throttling points that affect traffic patterns.
- **Streams are independent for loss recovery.** If a packet carrying data for Stream 4 is lost, only Stream 4 stalls while other streams keep flowing. This eliminates head-of-line blocking, but it also means that retransmission patterns in QUIC look different from TCP. A retransmission burst in QUIC affects only the relevant stream's data, not the entire connection.

2. Identifying Streams Based on Traffic Patterns

Since we cannot inspect the encrypted payload to see what kind of data is being transferred, we need to rely on observable metadata and statistical patterns to classify the *type* of traffic a QUIC flow

carries:

2.1 Video Streaming (YouTube, Netflix, etc.)

Video streaming over QUIC (specifically, adaptive bitrate streaming like DASH or HLS over HTTP/3) has some very distinctive characteristics:

Flow Duration: Long-lived. A video session can last anywhere from a few minutes to several hours. The underlying QUIC connection persists through this entire duration (and may survive network changes due to connection migration).

Packet Size Distribution: Bimodal. There will be clusters of large packets (typically 1200-1400 bytes for QUIC) carrying actual video/audio segment data, and smaller packets carrying ACKs, flow control updates, and request headers. The large packets dominate the volume.

Directionality: Heavily asymmetric. The vast majority of data flows from server to client (downstream). The client primarily sends ACKs, flow control updates, and periodic requests for the next segment. Upload-to-download byte ratio can be as extreme as 1:50 or even 1:100.

Burst Pattern: Adaptive bitrate players typically download video in chunks/segments (usually 2-10 seconds of playback each). The player requests a segment, receives a burst of data, buffers it, and then may pause briefly before requesting the next one. This creates a distinctive on-off pattern i.e. periodic bursts of high throughput followed by relative quiet. The periodicity roughly corresponds to the segment duration.

However, this pattern varies based on the player implementation:

- Some players use "buffer-based" algorithms that try to maintain a target buffer level. When the buffer is full, they slow down or stop fetching.
- Others use "throughput-based" algorithms that continuously fetch at a rate matched to available bandwidth.
- Modern players (like YouTube's) often use hybrid approaches.

Quality Adaptation Signatures: When the player switches between quality levels (e.g., from 720p to 1080p), the segment sizes change noticeably. A sudden increase in average packet burst size without a change in burst frequency could indicate an upward quality switch. Similarly, a drop in burst size could indicate a downward switch triggered by congestion.

Number of Streams: A video session typically uses relatively few concurrent streams. There might be separate streams for video segments, audio segments (if they are muxed separately), and perhaps manifest/playlist updates. But compared to web browsing, the number of parallel streams is small.

Inter-arrival Times: During a burst (segment download), packets arrive rapidly with very small inter-arrival times (often sub-millisecond). Between bursts, there may be longer gaps (hundreds of milliseconds to several seconds).

2.2 Web Browsing

Web browsing over HTTP/3 has a fundamentally different traffic shape:

Flow Duration: Short to medium. A page load might complete in 1-5 seconds, though the connection may be kept alive for subsequent navigations (HTTP/3 connection coalescing means multiple hostnames might share a single connection if they share a TLS certificate and resolve to the same IP).

Packet Size Distribution: Highly variable. HTML documents tend to be small to medium. CSS and JavaScript files vary widely. Images can be large. All of these are interleaved on different streams within the same connection. We see a wide spread of packet sizes rather than the bimodal distribution of video streaming.

Burst Characteristics: Web browsing tends to start with a "waterfall" pattern. The browser requests the HTML page, parses it, discovers embedded resources, and then fires off many parallel requests. This creates an initial burst of activity followed by a period where resources are downloading in parallel, followed by a taper-off. There is no periodicity. The pattern is more like a single burst (or a few cascading bursts for complex pages) followed by silence until the next user action.

Directionality: Still asymmetric (more downstream than upstream), but less extreme than video streaming. The upstream component includes HTTP request headers, POST data (form submissions), and ACKs. Upload-to-download ratio might be around 1:5 to 1:15 depending on the page.

Number of Streams: High. A modern web page can trigger 50-200+ HTTP requests, and many of them will be in flight concurrently. HTTP/3's multiplexing means these all share one connection, but the high degree of parallelism is a distinguishing feature.

Stream Priority: HTTP/3 supports priority hints (via the PRIORITY frame or the Priority header field). Browsers typically prioritize HTML and CSS over images and fonts. While we cannot see these priorities in encrypted traffic, they do influence the *order* and *timing* of data delivery, which subtly affects the packet-level pattern.

2.3 Voice over IP (VoIP) / Real-time Communication

VoIP and real-time communication over QUIC (WebTransport or custom protocols) have extremely distinctive patterns:

Packet Size Distribution: Very consistent and small. Voice codecs produce fixed-size frames at regular intervals. For example, Opus at 20ms framing produces packets that are typically between 40-150 bytes of payload, depending on the bitrate. With QUIC overhead added (headers, encryption), you might see UDP datagrams of around 80-200 bytes. The lack of variation in packet size is the clearest signal.

Timing Pattern: Extremely regular. Voice packets are generated at constant intervals, typically every 10, 20, or 30 milliseconds. This creates a metronomic inter-arrival pattern that is very different from the bursty patterns of video or web browsing. Even with network jitter, the *sending* pattern is regular, and statistical analysis of inter-arrival times will show a clear peak at the framing interval.

Directionality: Symmetric or near-symmetric. Both parties are sending and receiving voice data. The byte counts in each direction should be roughly equal (within maybe 1:1 to 1:2 ratio).

Flow Duration: Medium to long. A voice call might last a few minutes to an hour+. Unlike video streaming, there is no on-off pattern. The flow of packets is continuous for the duration of the call (unless silence suppression is used, in which case there are gaps when no one is speaking, but the packet rate drops rather than going to zero since comfort noise packets are typically sent).

Packet Rate vs. Throughput: The packet rate is high relative to the throughput. Sending 50 packets per second (20ms framing) that are each 100 bytes is only 40 kbps, but the packet rate is similar to what you might see in a moderate throughput bulk transfer. This high-packet-rate-low-throughput combination is characteristic of VoIP.

Sensitivity to Latency and Jitter: VoIP traffic is extremely latency-sensitive. This does not directly affect the traffic pattern we observe, but it is crucial for the policing section. The aggressive rate-limiting of VoIP traffic will cause severe quality degradation.

2.4 File Downloads / Uploads

Large file transfers (cloud storage sync, software updates, etc.) over HTTP/3 look like:

Packet Size Distribution: Dominated by maximum-size packets. Virtually all packets in the data-heavy direction will be at or near the MTU limit. Very few small packets apart from ACKs in the reverse direction.

Duration: Variable, but can be long for large files.

Burst Pattern: Sustained throughput rather than on-off bursts. The sender (or receiver, for uploads) is continuously transmitting at the maximum rate allowed by congestion control. This is a "bulk transfer" pattern.

Number of Streams: Typically one or very few. A single file download uses one stream. Even if multiple files are downloaded, they might be sequential rather than parallel.

Directionality: Extremely asymmetric. Almost all data in one direction with ACKs in the other.

2.5 Online Gaming

Gaming traffic over QUIC:

Packet Size: Small to medium, relatively consistent. Game state updates are small structured data.

Timing: Very regular, tied to the game's tick rate (e.g., 20, 30, 60, or 128 ticks per second for competitive games). This creates regular inter-arrival patterns similar to VoIP but potentially at different intervals.

Directionality: More symmetric than streaming but not perfectly so. The server sends more data (world state) than the client (player input).

Latency Sensitivity: Extremely high, even more so than VoIP in many cases.

2.6 Summary Table for Traffic Pattern Identification

Feature	Video Streaming	Web Browsing	VoIP	File Transfer	Gaming
Flow Duration	Long (min-hours)	Short-Medium (sec-min)	Medium-Long	Variable	Long
Pkt Size Dominant	Large (near MTU)	Variable	Small, consistent	Large (near MTU)	Small-Medium
Pkt Size Variance	Low (bimodal)	High	Very Low	Very Low	Low
Burst Pattern	Periodic on-off	Cascading waterfall	Continuous regular	Continuous sustained	Continuous regular
Directionality	Highly asymmetric (DL)	Asymmetric (DL)	Symmetric	Extremely asymmetric	Slightly asymmetric
Packet Rate	Medium	Bursty	High (relative to BW)	High	High
# Concurrent Streams	Few	Many	Very few	Very few	Few

2.7 Important Considerations and Edge Cases

Connection Coalescing: HTTP/3 allows multiple origins to share a single QUIC connection if they share a TLS certificate and IP address. This means what appears to be a single flow might carry traffic for multiple websites or services simultaneously. A video playing in one tab and a web page loading in another tab could share the same QUIC connection. This complicates classification because the traffic pattern becomes a blend of multiple application types.

Multiplexed Applications: Even within a single origin, different types of content are multiplexed. A YouTube page load involves the HTML page, CSS, JavaScript, thumbnails, API calls, and the video itself, all on the same connection. The traffic pattern transitions from "web browsing" during the initial page load to "video streaming" once playback begins.

CDN Behavior: Most QUIC traffic goes through CDNs like Google's, Cloudflare's, or Akamai's servers. Multiple different services might share the same CDN IP address, making server-IP-based classification unreliable. However, the SNI (Server Name Indication) from the handshake can help distinguish them.

VPN and Proxy Tunneling: When QUIC carries tunneled traffic (MASQUE, VPN), the observable pattern reflects the tunnel rather than the inner application.

Adaptive Behavior: Applications adapt to network conditions. A video player might behave like a bulk transfer when the buffer is empty (aggressive downloading) and switch to on-off when the buffer is full. The traffic pattern of the same application can change over the lifetime of a connection.

3. Packet Size Distribution and Burst Patterns

This section explores the statistical properties of QUIC traffic that are observable without decryption.

3.1 QUIC Packet Size Constraints

Minimum Size: QUIC Initial packets must be at least 1200 bytes (padded if necessary). This is mandated by RFC 9000 to prevent amplification attacks and to ensure path MTU is at least 1200 bytes. So during the handshake, we will always see at least some large packets.

Maximum Size: Determined by the path MTU. QUIC performs its own Path MTU Discovery (PMTUD) using DPLPMTUD (Datagram Packetization Layer PMTUD). The maximum UDP payload for QUIC is typically around 1200-1452 bytes on most Internet paths (1200 being the minimum guaranteed). Many implementations default to 1200 bytes initially and probe upward.

Coalesced Packets: Multiple QUIC packets can be packed into a single UDP datagram. This is common during the handshake - an Initial packet and a Handshake packet (and sometimes a 0-RTT packet) may be coalesced into a single large UDP datagram. Post-handshake, this is less common but still possible. This means the UDP datagram size might not directly reflect the size of individual QUIC packets.

3.2 Packet Size Distributions by Application Type

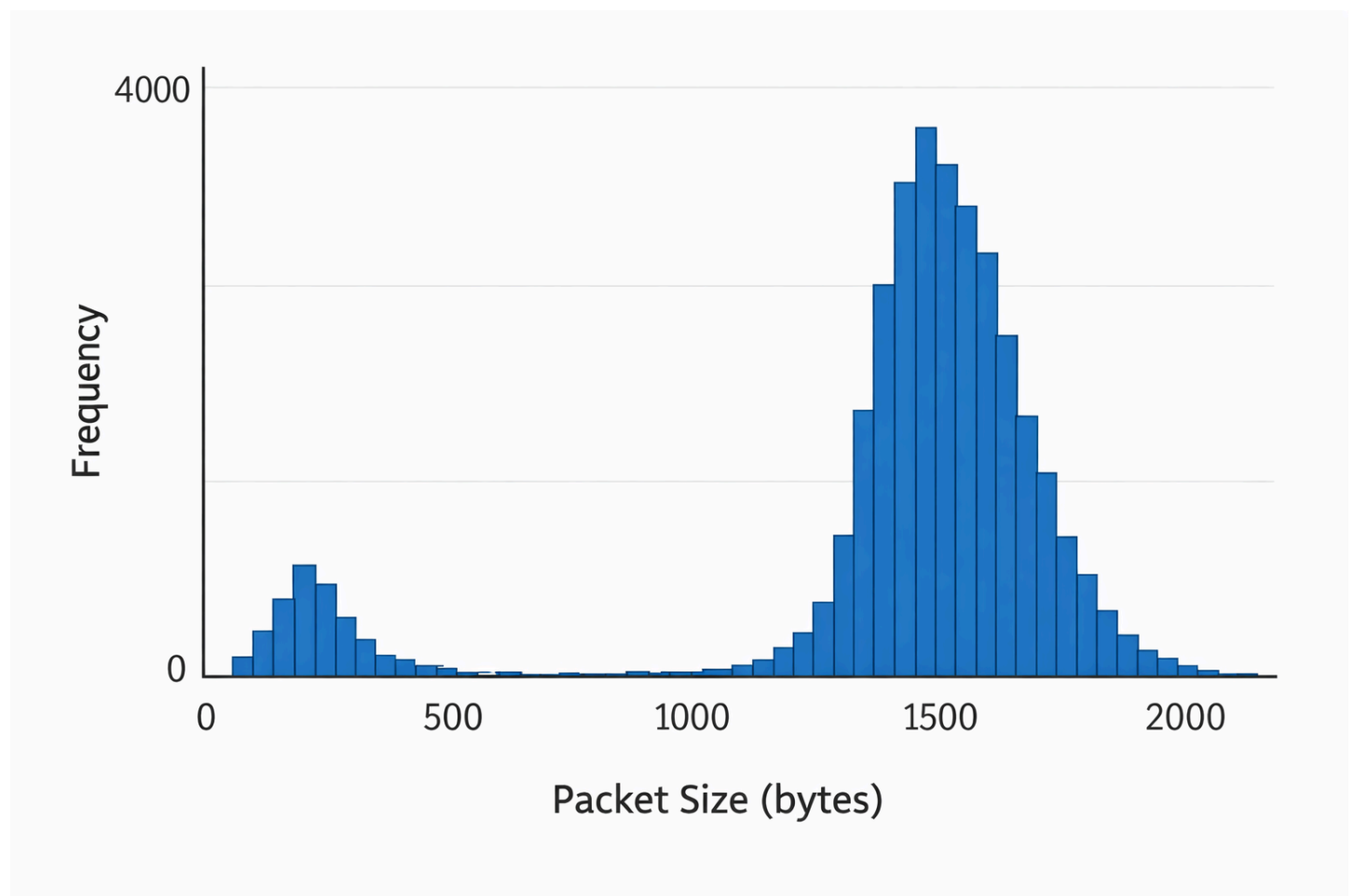
Video Streaming Distribution:

The distribution tends to be bimodal with two clear peaks:

- A peak near the MTU (1200-1400 bytes) representing video data segments
- A smaller peak at low sizes (40-100 bytes) representing ACKs and control traffic

The ratio of large to small packets tells us about the data transfer efficiency. For a high-bitrate video stream, the ratio heavily favors large packets.

If we plot a histogram of packet sizes for a video session, we would expect to see:



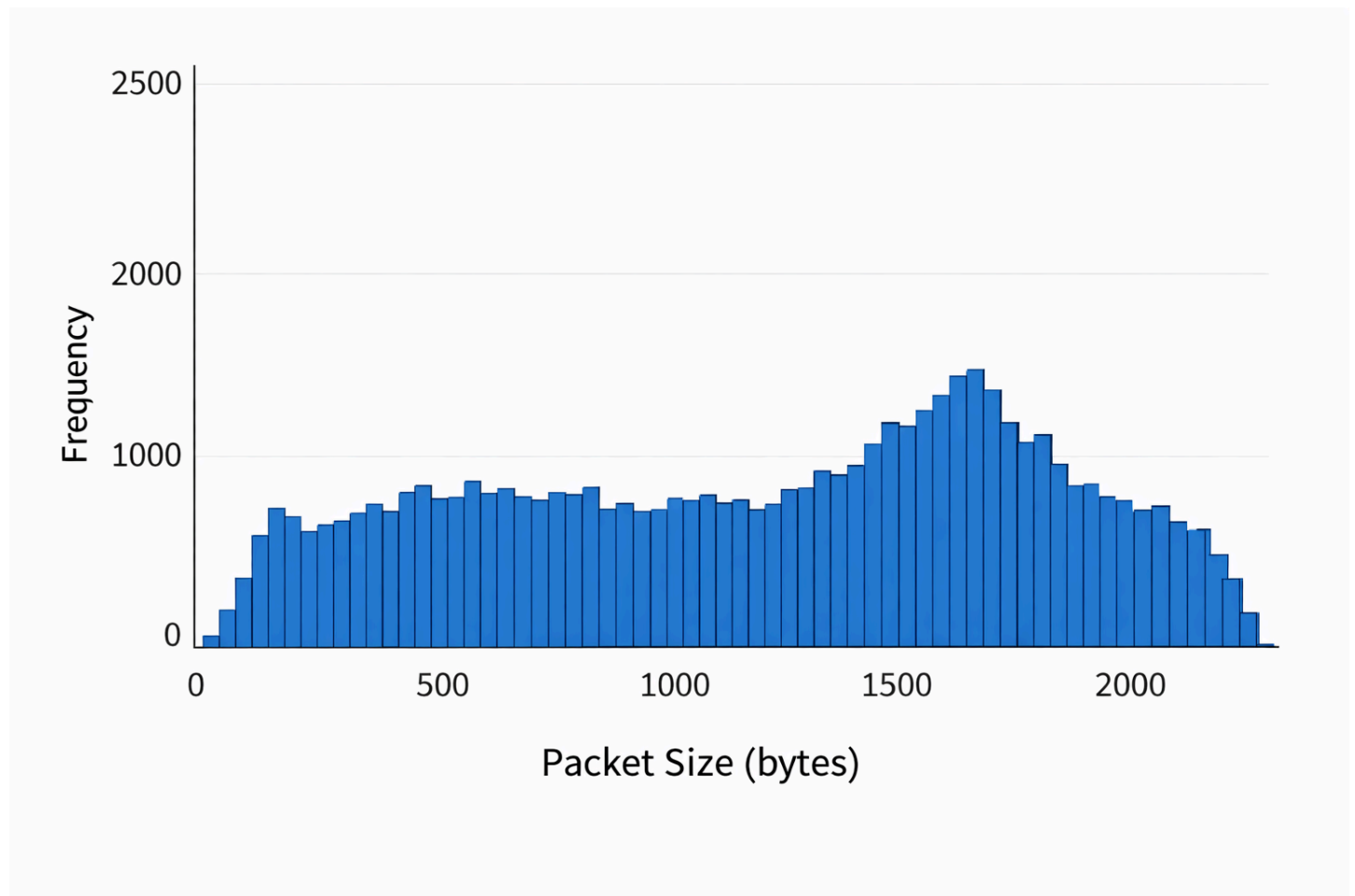
Web Browsing Distribution:

Much more spread out. There will be:

- Small packets for ACKs and small HTTP responses/headers

- Medium packets for HTML, CSS, small JavaScript files
- Large packets for images and large scripts

The distribution tends to be more uniform or multi-modal, with no single dominant peak. The variance is high.



VoIP Distribution:

Very narrow, single peak. Nearly all packets cluster tightly around one size (determined by the codec and framing). The standard deviation is very low.

3.3 Burst Pattern Analysis

A burst is a sequence of packets with inter-arrival times below some threshold. Common approaches use thresholds like 50ms or 100ms, if consecutive packets are less than this apart, they belong to the same burst.

Video Streaming Bursts:

- Burst size (in bytes): Corresponds to the video segment size. For a 2-second segment at 5 Mbps, that is roughly 1.25 MB per burst.
- Burst duration: Typically a few hundred milliseconds to a few seconds, depending on bandwidth.
- Inter-burst gap: Related to segment duration minus download time. If segments are 4 seconds and download takes 1 second, the gap is roughly 3 seconds. This gap shrinks as available

bandwidth decreases.

- Burst frequency: Semi-regular, tied to segment duration. Not perfectly periodic because of variable segment sizes and adaptive bitrate selection.

Web Browsing Bursts:

- Initial large burst during page load (many resources requested in parallel)
- Possibly secondary smaller bursts as lazy-loaded resources or async API calls trigger
- No regular periodicity
- Burst size varies widely between different page loads

VoIP Bursts:

- No real "bursts". The traffic is a continuous stream of regularly spaced small packets
- Can be viewed as one continuous burst for the entire call duration
- Alternatively, each individual packet is a "burst" of one if analyzed at a fine granularity

3.4 Statistical Features Worth Computing

For our classification system, the following statistical features computed from packet-level data over time windows can be useful:

First-order statistics (per time window):

- Mean packet size
- Standard deviation of packet sizes
- Min and max packet size
- Median and various percentiles (25th, 75th, 95th)
- Packet count
- Total bytes
- Mean inter-arrival time
- Standard deviation of inter-arrival times
- Bytes ratio (upstream bytes / downstream bytes)
- Packet ratio (upstream packets / downstream packets)

Second-order statistics:

- Autocorrelation of packet sizes (reveals periodicity)
- Autocorrelation of inter-arrival times
- Coefficient of variation of inter-arrival times (standard deviation / mean)
- Burstiness metric (variance-to-mean ratio of packet counts per sub-interval)

Sequence-based features:

- Distribution of packet size sequences (n-grams of quantized packet sizes)

- First N packet sizes and directions (useful for early classification)
- Run lengths of consecutive upstream/downstream packets

These features should be computed over an appropriate time window. Too short, and the statistics are unreliable. Too long, and you lose the ability to detect changes in traffic type mid-connection. A window of 5-30 seconds with sliding overlap is a reasonable starting point.

3.5 The First Few Packets of QUIC

The first few packets of a QUIC connection are particularly informative. The handshake itself (Initial, Handshake, and early application data) follows a predictable pattern. Specifically:

1. Client → Server: Initial (1200+ bytes, padded)
2. Server → Client: Initial + Handshake (possibly coalesced, often large)
3. Client → Server: Handshake + early short header packets
4. Then application data begins

The *timing and sizes of the first 5-10 application data packets* after the handshake can already provide strong signals about the application type. For instance, if the first application data packets from the server are a few small-to-medium packets (HTTP response headers, HTML) followed by many large packets (images, scripts), it looks like web browsing. If the first data is a moderate burst of small upstream packets (HTTP requests for video manifest and initial segments) followed by sustained large downstream packets, it looks like video streaming.

4. QUIC Headers

This provides a byte-level analysis of QUIC packet headers, focusing specifically on which fields are visible to a network observer (middlebox, firewall, traffic classifier) and what information can be extracted.

4.1 The Two Header Forms

QUIC has two header forms, distinguished by the most significant bit (Header Form bit) of the first byte:

- **Long Header (bit = 1):** Used during connection establishment (Initial, Handshake, 0-RTT, Retry packets). Contains more metadata and is more amenable to inspection.
- **Short Header (bit = 0):** Used for post-handshake application data (1-RTT packets). Minimal header, optimized for size.

4.2 Long Header Format

```
Long Header Packet {  
    Header Form (1) = 1,  
    Fixed Bit (1) = 1,  
    Long Packet Type (2),  
    Type-Specific Bits (4),  
    Version (32),  
    Destination Connection ID Length (8),  
    Destination Connection ID (0..160),  
    Source Connection ID Length (8),  
    Source Connection ID (0..160),  
    Type-Specific Payload (...)  
}
```

Byte 0 (First Byte):

- Bit 7 (MSB): Header Form = 1 (indicates long header)
- Bit 6: Fixed Bit = 1 (QUIC invariant. It must be 1 in all QUIC v1 packets; this is used to distinguish QUIC from some other UDP protocols)
- Bits 5-4: Long Packet Type:
 - 00 = Initial
 - 01 = 0-RTT
 - 10 = Handshake
 - 11 = Retry
- Bits 3-0: Type-Specific Bits. For Initial, 0-RTT, and Handshake packets, these are: 2 reserved bits + 2 bits encoding the Packet Number Length (but these bits are protected/encrypted, so a passive observer cannot read them directly without decrypting the header protection). For Retry, these are unused.

Bytes 1-4: Version (32 bits):

This is the QUIC version number. This field is always in the clear and is a QUIC invariant (present in all versions of QUIC). Key version values:

- 0x00000001 : QUIC v1 (RFC 9000)
- 0x6b3343cf : QUIC v2 (RFC 9369)
- 0x00000000 : Version Negotiation (special)
- 0xff0000xx : Draft versions (e.g., 0xff00001d for draft-29)
- Various "greased" versions used for version negotiation resistance

The version field is extremely useful for identifying QUIC traffic and distinguishing it from arbitrary UDP. If a UDP datagram's first byte has bit 7 set and bytes 1-4 match a known QUIC version, it is very likely QUIC.

Bytes 5: DCID Length (8 bits):

Length of the Destination Connection ID that follows. Can be 0-20 bytes.

Next DCID Length bytes: Destination Connection ID:

The connection ID used by the *receiver* to identify which connection this packet belongs to. During the handshake, the client generates an initial DCID that may be random. The server then provides its own preferred connection IDs via NEW_CONNECTION_ID frames. CIDs are not confidential, they are always visible.

Next byte: SCID Length (8 bits):

Length of the Source Connection ID.

Next SCID Length bytes: Source Connection ID:

The connection ID chosen by the sender.

After the CIDs, the format diverges by packet type:**For Initial Packets:**

- Token Length (variable-length integer)
- Token (0..N bytes): Used for address validation (Retry token or NEW_TOKEN token). Opaque to observers.
- Length (variable-length integer): Length of the remaining packet including the Packet Number and encrypted payload
- Packet Number (1-4 bytes): **Header-protected** (encrypted). We can derive the header protection key for Initial packets because the Initial encryption keys are deterministic from the DCID, but doing so requires active computation.
- Payload: **Encrypted** with Initial keys (derived from DCID + version-specific salt)

For Handshake Packets:

- Length
- Packet Number (header-protected)
- Payload (encrypted with Handshake keys derived from TLS handshake)

For 0-RTT Packets:

- Length
- Packet Number (header-protected)
- Payload (encrypted with 0-RTT keys, which require the pre-shared key)

For Retry Packets:

- Retry Token (variable length)
- Retry Integrity Tag (128 bits)

- No packet number, no encrypted payload

4.3 Short Header Format

```
Short Header Packet (1-RTT) {
  Header Form (1) = 0,
  Fixed Bit (1) = 1,
  Spin Bit (1),
  Reserved Bits (2),
  Key Phase (1),
  Packet Number Length (2),
  Destination Connection ID (variable),
  Packet Number (8..32),      ← header-protected
  Payload (..)                ← encrypted
}
```

Byte 0:

- Bit 7: Header Form = 0 (short header)
- Bit 6: Fixed Bit = 1
- Bit 5: **Spin Bit** . The spin bit is a single bit that alternates (spins) once per round trip. By observing the spin bit changes, a passive observer can estimate the RTT of the connection. This was explicitly designed as an observable signal for network operators.
- Bits 4-3: Reserved Bits (encrypted via header protection, not readable without key)
- Bit 2: Key Phase, Indicates which set of application keys was used. Changes when key update occurs. **Also header-protected.**
- Bits 1-0: Packet Number Length, Encodes the length of the Packet Number field (1-4 bytes). **Header-protected.**

After Byte 0:

- Destination Connection ID: The observer needs to know the CID length from the handshake (it is not encoded in the short header). This is a significant challenge. if a middlebox starts observing mid-connection, it does not know the CID length.
- Packet Number: Header-protected
- Payload: Encrypted

4.4 What Is Visible vs. What Is Not

Let me consolidate this because it is the crux of our analysis capability:

Always Visible (in every packet):

- Whether the packet uses long or short header (1 bit)

- Fixed Bit (1 bit)
- UDP source and destination ports
- UDP datagram size
- IP addresses (source and destination)
- Packet timestamp (from capture)

Visible in Long Header Packets (handshake):

- QUIC version
- Packet type (Initial/Handshake/0-RTT/Retry)
- Destination Connection ID (and its length)
- Source Connection ID (and its length)
- Token (in Initial packets, present but opaque)
- Length field

Visible in Short Header Packets (post-handshake):

- Spin Bit (for RTT estimation)
- Destination Connection ID (if the observer knows the length from observing the handshake)

That is essentially it for reliably readable fields

Extractable from Initial Packets with Computation:

Since Initial packet encryption uses keys derived deterministically from the Destination Connection ID and a version-specific salt (published in the RFC), it is technically possible for a middlebox to:

- Derive the Initial secret
- Remove header protection to read the Packet Number
- Decrypt the Initial packet payload
- Parse the CRYPTO frames inside, which contain the TLS ClientHello/ServerHello

From the TLS ClientHello, we can extract:

- **Server Name Indication (SNI):** The hostname the client is connecting to (e.g., "www.youtube.com").
- **ALPN (Application-Layer Protocol Negotiation):** What application protocol is being requested (e.g., "h3" for HTTP/3, "doq" for DNS over QUIC).
- **Supported QUIC Versions** (via transport parameters)
- **TLS extensions** including supported cipher suites, key shares, etc.

While Initial encryption is "easily" breakable (since the key material is derived from public information), doing this at line rate on a high-throughput link requires non-trivial computational resources. It involves HKDF key derivation, AES-128-ECB for header protection removal, and AES-

128-GCM for payload decryption. This is feasible but adds processing cost that must be considered in the policing architecture.

4.5 Connection IDs

Connection IDs deserve special attention because they are our primary flow-tracking mechanism:

- **CIDs are not static.** An endpoint can issue new CIDs using `NEW_CONNECTION_ID` frames and retire old ones using `RETIRE_CONNECTION_ID` frames. This means a single logical connection may use multiple different CIDs over its lifetime.
- **CID rotation for privacy.** When connection migration occurs (e.g., a phone switches from Wi-Fi to cellular), the migrating endpoint uses a new CID to prevent linking the old and new paths. From a middlebox perspective, the flow "disappears" from one 5-tuple and "appears" on another with no obvious linkage.
- **CID length is variable** (0-20 bytes). A CID length of 0 is valid and means no CID is used (the connection is identified solely by the UDP 4-tuple). Some server deployments use CID length of 0 for short header packets, which saves bytes but prevents connection migration.
- **Server CID encoding:** Many server implementations encode routing information in the CID (e.g., which backend server should handle the connection). This is formalized in draft-ietf-quickload-balancers. As external observers, we cannot decode this without knowing the encoding scheme, but the structure/pattern of CIDs might reveal the server implementation.

4.6 The Spin Bit - RTT Measurement

The spin bit is the only explicitly designed passive measurement signal in QUIC's short header:

How it works: The client sets the spin bit to the opposite value of the spin bit it received from the server. The server does the same. This creates a "spin" signal that alternates once per RTT. A passive observer watching the spin bit transitions can measure the RTT by timing the interval between consecutive transitions.

Limitations:

- Only one RTT sample per round-trip (low resolution for short connections)
- Can be unreliable during loss or reordering
- Implementations can deliberately randomize the spin bit to prevent measurement (privacy concern). RFC 9000 allows this.
- Only meaningful if you can observe both directions of the flow (i.e., you are on the path for both directions)

For our project: The spin bit gives us a free latency estimate, which could be useful for classification (VoIP connections tend to be on low-latency paths, while video streaming is more tolerant) and for adaptive policing (detecting congestion).

4.7 QUIC Invariants (RFC 8999)

RFC 8999 defines the properties that are guaranteed to remain stable across all versions of QUIC. These are the only things a version-agnostic middlebox can rely on:

- 1. The first byte has a Header Form bit (bit 7) and a Fixed Bit (bit 6, always 1 for QUIC v1).
- 2. Long header packets have Version, DCID Length, DCID, SCID Length, SCID at fixed positions.
- 3. Version Negotiation packets have a specific format.
- 4. Short header packets have the DCID immediately after the first byte, but the length is not encoded in the header.

Everything else like spin bit position, packet types, encryption schemes can change between versions.

6. Encrypted Details Extractable from QUIC Headers

This section consolidates and expands on what information can be obtained despite QUIC's encryption.

6.1 Directly Observable Metadata (No Decryption)

Information	Source	Availability
5-tuple (IPs, ports, protocol)	IP + UDP headers	Always
Packet size	UDP length	Always
Packet timing	Capture timestamp	Always
Header form (long/short)	First byte, bit 7	Always
Fixed bit	First byte, bit 6	Always
QUIC version	Long header, bytes 1-4	Handshake only
Packet type	Long header, bits 5-4 of byte 0	Handshake only
Connection IDs	Long header: explicit; Short header: implicit	Always (but length unknown for short headers)
Spin bit	Short header, bit 5 of byte 0	Post-handshake

Information	Source	Availability
Number of coalesced packets	By parsing multiple QUIC packets within one UDP datagram	Handshake (primarily)

6.2 Information from Initial Packet Decryption

As discussed, Initial packets can be decrypted because their keys are derived deterministically. The steps conceptually are:

1. Extract the Destination Connection ID from the Initial packet.
2. Derive the Initial secret using HKDF-Extract with a version-specific salt and the DCID.
3. Derive client/server Initial keys and IVs using HKDF-Expand-Label.
4. Use the header protection key (AES-ECB) to unmask the protected header bits and packet number.
5. Use the packet protection key (AES-128-GCM or ChaCha20-Poly1305) to decrypt the payload.

From the decrypted Initial payload, we find CRYPTO frames containing TLS handshake messages. Specifically:

From ClientHello (client's Initial):

- **SNI (Server Name Indication):** The target hostname. Critical for domain-based classification.
- **ALPN values:** Requested application protocols. "h3" = HTTP/3, "h3-29" = HTTP/3 draft-29, "doq" = DNS over QUIC, "hq-interop" = HTTP/0.9 over QUIC (testing), etc.
- **QUIC Transport Parameters extension:** Contains the client's proposed settings:
 - `initial_max_streams_bidi` and `initial_max_streams_uni` — how many concurrent streams the client wants
 - `initial_max_data` — connection-level flow control window
 - `initial_max_stream_data_bidi_local`, `_remote`, `_uni` — per-stream flow control windows
 - `max_idle_timeout` — how long the connection can be idle before closing
 - `max_udp_payload_size` — maximum UDP payload the client will accept
 - `active_connection_id_limit` — how many connection IDs the client can track
 - `initial_source_connection_id` — echoes the SCID from the Initial packet

These transport parameters are interesting because they reveal the client implementation's preferences. A browser establishing a connection for web browsing might request many parallel streams (high `initial_max_streams_bidi`), while a VoIP application might request very few.

- **Supported TLS versions and cipher suites:** Can help fingerprint the client implementation.
- **Key Share extension:** The cryptographic key material for the handshake.
- **Various other TLS extensions** that can contribute to fingerprinting (supported groups, signature algorithms, etc.).

From ServerHello + EncryptedExtensions (server's Initial/Handshake):

- Server's chosen ALPN (confirms the application protocol)
- Server's QUIC Transport Parameters (similar set as above, from server's perspective)
- Server's TLS certificate chain (in the Handshake packets, not Initial — these use Handshake encryption keys, which are NOT derivable by a passive observer)

What we CANNOT get even from Initial decryption:

- The actual application data (this uses 1-RTT keys, derived from the TLS key exchange)
- The TLS server certificate (encrypted in Handshake packets with Handshake-level keys)
- 0-RTT data (encrypted with keys from a previous connection's resumption secret)

6.3 TLS Fingerprinting of QUIC Clients

Even beyond extracting specific fields, the structure of the ClientHello itself serves as a fingerprint for the client application. Different browsers and QUIC implementations produce distinctly different ClientHellos. This technique, adapted from JA3 fingerprinting for TLS-over-TCP, creates a hash of the following ClientHello properties:

- TLS version
- List of cipher suites
- List of extensions (in order)
- List of elliptic curves (supported groups)
- List of EC point formats

The resulting hash (sometimes called a JA3 hash, or JA4 in newer variants) can identify specific browser versions, applications, or QUIC libraries. For example:

- Chrome's QUIC stack produces a distinctive ClientHello
- Firefox's QUIC (using neqo) produces a different one
- curl with quiche produces yet another

This fingerprinting does not tell us what the *content* is, but it tells us *what application is making the connection*, which is a strong indirect signal. A YouTube connection from Chrome looks the same at the ClientHello level regardless of whether the user is watching a video or browsing thumbnails — but we know it is Chrome talking to YouTube, which narrows the classification space considerably.

6.4 0-RTT Considerations

When a client has previously connected to a server, it may have a resumption token (PSK — Pre-Shared Key) that allows 0-RTT data. In this case:

- The client sends 0-RTT packets alongside the Initial packet
- 0-RTT data is encrypted with keys derived from the PSK, which a passive observer does NOT have

- We can observe that 0-RTT is being used (by the presence of 0-RTT packet type in the long header), but we cannot decrypt it
- The fact that 0-RTT is used tells us this is a repeat connection — the client has connected to this server before. This is a weak but sometimes useful signal.

6.5 Connection Migration Implications

When a QUIC connection migrates (changes IP address or port — common on mobile devices switching between Wi-Fi and cellular):

- The new packets use a different CID (typically) and a different 4-tuple
- Without additional context, the middlebox sees the old flow "die" and a new flow "appear"
- There is no reliable way to link the old and new flows passively unless:
 - The middlebox tracked the set of CIDs exchanged during the original handshake (from `NEW_CONNECTION_ID` frames, which are encrypted)
 - The middlebox uses timing correlation (old flow stops, new flow appears to the same server around the same time)

This is a significant challenge for long-lived flow tracking and policing. If we are rate-limiting a video stream and the user's phone switches networks, we lose the classification state and have to re-classify the "new" flow from scratch.

7. Payload Pattern Analysis

Even though the QUIC payload is encrypted, there are patterns in the encrypted data that can reveal information. This section examines what can be inferred.

7.1 Why Encrypted Payloads Are Not Perfectly Opaque

Encryption protects the *confidentiality* of data — an observer cannot read the content. But encryption does NOT hide:

- **The length of the plaintext** (with minor caveats for padding): The ciphertext length reveals the plaintext length (plus a fixed authentication tag overhead, which is 16 bytes for AES-GCM and ChaCha20-Poly1305). If the application sends a 500-byte message, the encrypted payload will be approximately 500 + 16 bytes.
- **The timing of messages**: When data is sent is not hidden by encryption.
- **The direction of messages**: Who is sending to whom is visible.

These three pieces of information — size, timing, direction — form the basis of traffic analysis, and they are fundamentally not protected by transport-level encryption.

7.2 QUIC Frame Padding and Its Effect

QUIC includes a PADDING frame type (type 0x00) that allows senders to inflate packet sizes. Some implementations use padding for various purposes:

- **Initial packet padding:** Required by spec (must be at least 1200 bytes)
- **PMTUD probing:** Padded packets to test path MTU
- **Traffic analysis resistance:** Some privacy-focused implementations might pad all packets to fixed sizes to obscure content length

However, in practice, most QUIC implementations do NOT perform systematic payload padding beyond what the spec requires. Chrome, for example, does not pad regular data packets. This means the encrypted packet sizes closely reflect the actual data sizes, which is good for our classification purposes.

If future implementations adopt aggressive padding, our size-based classification would be degraded. This is something to monitor but is not a current practical concern for most traffic.

7.3 Encrypted Payload Size as a Content Indicator

HTTP/3 Header-Only Responses: Small encrypted payloads (roughly 50-200 bytes) after a request likely contain only HTTP response headers (e.g., redirects, 304 Not Modified, small API responses). These are common in web browsing but rare in video streaming once playback has started.

Video Segment Data: Large encrypted payloads consistently near MTU size indicate bulk data transfer. For video streaming, consecutive MTU-sized packets from server to client correspond to video segment delivery.

QPACK Updates: Very small payloads on the QPACK streams. These are interspersed with other data.

Flow Control and ACKs: When QUIC sends ACK frames without piggybacked data, the packets are small. The ACK frame encodes acknowledgment ranges and may vary in size depending on how much reordering has occurred.

7.4 Statistical Patterns in Encrypted Traffic

Entropy analysis of the encrypted payload is NOT useful for classification. Properly encrypted data is indistinguishable from random, so all encrypted QUIC payloads have roughly the same entropy (maximum). This is unlike some older encryption schemes where patterns could leak through.

However, **sequential patterns in packet sizes** are informative:

- **Video adaptive streaming sequence:** A characteristic pattern is: small upstream packet (segment request) → pause → burst of large downstream packets (segment data) → small downstream packet (end of response) → repeat. The number of large packets in each burst corresponds to the segment size, and changes in burst size correlate with quality switches.
- **Web page load sequence:** Characteristic pattern is: one or a few upstream packets (initial request) → small-medium downstream packets (HTML) → burst of small upstream packets (parallel resource requests) → interleaved medium-to-large downstream packets (various resources) → tapering off.
- **VoIP sequence:** Alternating small packets in both directions with metronomic regularity.

7.5 Encrypted Traffic Classification Techniques (Conceptual)

The research community has extensively studied encrypted traffic classification, and many techniques developed for TLS-over-TCP apply (with modifications) to QUIC:

Flow-level statistical classification: Compute statistical features (as described in Section 4.4) over the flow and use a classifier (Random Forest, SVM, neural network) to categorize flows. This has been shown to achieve >90% accuracy for broad categories (video, web, VoIP, file transfer) even for QUIC traffic.

Early classification: Classify based on the first N packets only. The advantage is low latency — we can make a classification decision within the first second of a flow. The disadvantage is lower accuracy, especially for flows that start similarly (e.g., both video streaming and web browsing start with what looks like an HTTP request-response).

Time-series approaches: Treat the sequence of (timestamp, size, direction) tuples as a time series and use sequence classifiers (RNN, LSTM, 1D-CNN, or even transformer-based models). These can capture temporal patterns that simple statistical features miss.

Clustering for unknown traffic: Unsupervised learning (K-means, DBSCAN) on flow features to discover clusters of similar traffic. Useful for identifying new application types that we have not trained on.

7.6 Practical Considerations for Payload Analysis

- **We never need to decrypt the payload** for statistical analysis. All the features we use are metadata (sizes, timing, direction). This is important both practically (we cannot decrypt 1-RTT data) and legally/ethically (we are not performing DPI on content).
- **Sampling may be necessary** at high link speeds. Analyzing every packet of every flow on a 10 Gbps link is computationally expensive. Sampling strategies (e.g., analyzing the first 50 packets of each flow for classification, then only tracking aggregate statistics) can reduce cost while maintaining accuracy.

- **Ground truth for training/validation** is hard to obtain. If we use machine learning for classification, we need labeled training data (we need to know the true application type for each flow). This typically requires controlled experiments where we generate known traffic types.

8. Flow Duration Analysis — Finding Long-Lived Flows

8.1 Why Flow Duration Matters

Flow duration is one of the most discriminative features for traffic classification:

- **Video streaming:** Tens of minutes to hours. If a flow has been active for 30 minutes with sustained data transfer, it is almost certainly streaming media.
- **Web browsing:** Seconds to a few minutes per page load. The QUIC connection may persist longer due to keep-alive, but active data transfer is short.
- **VoIP:** Minutes to an hour+, with continuous bi-directional data.
- **File transfer:** Seconds to minutes for typical files; longer for very large files.
- **DNS over QUIC:** Milliseconds to seconds.

8.2 Defining "Flow Duration" for QUIC

This is trickier than it sounds for QUIC:

A QUIC connection is not a UDP flow. A single QUIC connection may span multiple UDP 5-tuples due to connection migration. If we track by UDP 5-tuple, we may underestimate connection duration.

Idle periods: A QUIC connection can be idle (no data packets exchanged) for extended periods while remaining "alive." HTTP/3 connections are often kept open for minutes or even hours in case the user returns to the same website. During idle periods, only occasional keep-alive probes (if configured) are exchanged. Should we count idle time as part of the flow duration? For classification purposes, it probably makes more sense to track "active flow duration" (total time during which data is being exchanged) separately from "connection duration."

Connection reuse: HTTP/3 uses persistent connections. A single QUIC connection might serve a page load, then sit idle for 30 seconds, then serve another page load. Each "activity period" within the connection should potentially be classified independently.

8.3 Measuring Flow Duration

Start of flow: The first Initial packet (SYN equivalent). This is clearly identifiable by the long header with Initial packet type.

End of flow: This is harder. Options include:

- **CONNECTION_CLOSE:** QUIC has an explicit connection close mechanism, but the CONNECTION_CLOSE frame is inside an encrypted packet, so we cannot observe it. We can only infer closure from the cessation of packets.
- **Idle timeout:** If no packets are exchanged for a period exceeding the idle timeout (negotiated via transport parameters — typically 30 seconds to several minutes), the connection is considered closed.
- **Last packet:** Simply note the timestamp of the last packet seen for the flow. The difference between first and last packet is the observed duration.

For practical purposes, defining a flow as "ended" after an inactivity period (e.g., 30 or 60 seconds of no packets) is the most reliable approach.

8.4 Activity Patterns Within Long-Lived Flows

For long-lived connections, the *pattern of activity* within the connection is more informative than the raw duration:

Continuous activity: Data packets flowing steadily for the entire duration → video streaming, file transfer, or VoIP.

Intermittent activity: Bursts of activity separated by idle periods → web browsing with keep-alive, or interactive applications.

Gradual ramp-up then sustained: Initial burst (handshake + page load) followed by sustained throughput → video streaming that starts with a web page load and transitions to video playback.

One useful metric is the **activity ratio**: (time periods with data transfer) / (total connection duration). A ratio near 1.0 indicates continuous activity (streaming, VoIP). A ratio below 0.5 indicates intermittent use (browsing, interactive apps).

8.5 Connection Migration and Flow Duration Tracking

As discussed in Section 6.5, connection migration breaks the UDP 5-tuple tracking. For the purposes of our project, we have a few options:

1. **Accept fragmented tracking:** Track by UDP 5-tuple and accept that migrated flows will appear as separate shorter flows. This is the simplest approach and may be acceptable if migration is

rare in the target network.

2. **CID-based tracking:** Track flows by Connection ID rather than 5-tuple. This works until the CIDs are rotated (which typically happens during migration, defeating the purpose).
3. **Heuristic linking:** If a flow to a specific server IP dies and a new flow to the same server IP appears from the same client subnet within a short window, probabilistically link them. This is error-prone but may be sufficient for aggregate policing.

For our initial prototype, option 1 is probably the most practical starting point.

9. Upload / Download Pattern Analysis

9.1 Directional Asymmetry as a Classification Signal

The ratio of upstream (client → server) to downstream (server → client) traffic is a strong signal for traffic type. We can measure this in terms of bytes, packets, or both.

Byte ratio (upstream bytes / downstream bytes):

- Video streaming: ~0.01 to 0.05 (extremely download-heavy)
- Web browsing: ~0.05 to 0.20
- VoIP: ~0.5 to 1.5 (roughly symmetric)
- File upload: >1.0 (upload-heavy, which is distinctive because most traffic is download-heavy)
- File download: ~0.01 to 0.05

Packet ratio (upstream packets / downstream packets):

- Video streaming: ~0.1 to 0.3 (more upstream packets than the byte ratio suggests, because upstream packets are small ACKs)
- Web browsing: ~0.2 to 0.5
- VoIP: ~0.5 to 1.5
- File download: ~0.1 to 0.3

The *difference* between the byte ratio and the packet ratio reveals the asymmetry in packet sizes. For video streaming, the packet ratio is much higher than the byte ratio because the client sends many small ACK packets for every large data packet received. For VoIP, both ratios are close to 1.0.

9.2 Upload-Specific Patterns

Upload-heavy QUIC traffic (file uploads, video uploading to platforms like YouTube, cloud backup) has distinctive features:

- Large packets predominantly from client to server (reversed from the typical direction)
- Server sends mostly ACKs and flow control updates
- Congestion control behavior is driven by the client (sender), so the upload rate adapts to the client's congestion window
- Upload connections often show slower ramp-up compared to downloads because client devices may have lower uplink bandwidth (especially on residential and mobile networks)

9.3 Bidirectional Patterns and Concurrent Streams

For applications that use QUIC bidirectionally (like real-time collaboration tools or WebSocket-over-HTTP/3):

- Both directions carry substantive data (not just ACKs)
- Data in both directions appears quasi-simultaneously
- Packet sizes in both directions may be similar (if both sides are sending similar content) or different (if one side sends text and the other sends media)

An interesting edge case is video conferencing, which combines:

- Upload: camera/microphone data (moderate bandwidth, regular timing)
- Download: other participants' camera/microphone data (higher bandwidth, from multiple participants)
- Additional downloads: screen sharing (bursty, high bandwidth when active)
- Background: signaling messages (infrequent, small)

The resulting traffic pattern is complex and does not fit neatly into any single category. Classification systems may need a separate category for "video conferencing" rather than trying to classify it as VoIP or video streaming.

9.4 Temporal Evolution of Directionality

The upload/download balance often changes over the lifetime of a connection:

Phase 1 — Handshake: Roughly symmetric (Initial ↔ Initial, Handshake ↔ Handshake)

Phase 2 — Request: Upstream-heavy (client sending HTTP requests)

Phase 3 — Response: Downstream-heavy (server sending data)

Phase 4 — Steady state: Depends on the application

For video streaming specifically:

- Phase 2 is very brief (a few request packets)
- Phase 3 dominates (continuous video data download)
- Periodically, small upstream bursts (new segment requests) interrupt the download stream

For web browsing:

- Phases 2 and 3 interleave as multiple resources are requested and received
- The overall pattern is a cascade of request-response exchanges

Analyzing how directionality evolves over time windows (e.g., computing the byte ratio in sliding 5-second windows) can reveal application behavior that a single aggregate ratio misses.

10. Traffic Simulation Tools

To develop and test our classification and policing system, we need the ability to generate controlled QUIC traffic of known types. This section reviews the available tools.

10.1 QUIC-Capable Client/Server Implementations

Google's quiche (C++):

Google's production QUIC implementation used in Chrome and Google servers. The standalone tools (`quic_client` and `quic_server`) can be built and used for testing. Very feature-complete and closely represents real-world Google QUIC traffic. However, it is tightly integrated into Chromium's build system, which makes standalone building non-trivial.

Cloudflare's quiche (Rust):

Confusingly also named "quiche" but a completely separate implementation. Written in Rust with C bindings. Provides command-line client and server tools. Easier to build and use than Google's version. Used by Cloudflare's production infrastructure, curl (as a backend), and nginx (via cloudflare's fork). Well-documented and actively maintained.

ngtcp2 (C):

A standalone QUIC library that uses a pluggable TLS backend (typically OpenSSL or BoringSSL or wolfSSL). Comes with example client and server. Clean, well-structured code. Supports QUIC v1, v2. Good for generating standards-compliant traffic.

aioquic (Python):

A pure Python QUIC implementation built on asyncio. Easy to use for scripting and prototyping. Comes with example client and server. Performance is limited (it is Python), so not suitable for high-throughput testing, but excellent for functional testing, protocol understanding, and generating small-scale traffic samples. Its simplicity makes it ideal for our initial exploration and prototyping phase.

Quinn (Rust):

A Rust QUIC implementation. Robust and performant. Has a growing ecosystem. Can be used to build custom client/server tools.

msquic (C):

Microsoft's QUIC implementation. Used in Windows, .NET, and SMB over QUIC. Cross-platform (Linux, Windows, macOS). Well-optimized and comes with performance testing tools. Has a `quicinterop` tool that can generate various traffic patterns.

picoquic (C):

Written by one of the QUIC RFC authors (Christian Huitema). Designed for experimentation and testing. Includes a demo server/client and supports many test scenarios.

10.2 Traffic Generation Tools

curl (with HTTP/3 support):

curl supports HTTP/3 when compiled with a QUIC backend (quiche, ngtcp2, or msquic). This means we can simply use curl to make HTTP/3 requests to any HTTP/3-enabled server (e.g., Google, Cloudflare, Facebook). This is the easiest way to generate real QUIC traffic.

Example usage (conceptual): `curl --http3 https://www.google.com`

Web browsers (Chrome, Firefox, Edge):

The most realistic QUIC traffic generators, because they produce exactly the traffic patterns that users create. Chrome has had QUIC support for years and uses it by default for Google services and many other sites. Firefox supports HTTP/3 as well. Running browsers with automated scripting (Selenium, Playwright, Puppeteer) allows generating reproducible browsing sessions.

Chrome's `chrome://net-internals/#quic` page shows active QUIC sessions, and Chrome can export QUIC event logs (via `--log-net-log`) for detailed analysis.

iperf with QUIC (experimental):

Some forks/versions of iperf support QUIC. Useful for generating bulk transfer traffic at controlled rates.

Custom traffic generators:

For specific traffic patterns (e.g., VoIP-like periodic small packets), we may need to write custom generators using one of the QUIC libraries (aioquic is ideal for this due to Python's ease of use).

10.3 Traffic Capture and Replay

Wireshark:

Full QUIC protocol dissection support. Can decrypt QUIC traffic if provided with the TLS key log file (set `SSLKEYLOGFILE` environment variable before starting the browser/client). Extremely valuable for understanding packet-level behavior and validating our parser's output.

tcpdump / tshark:

Command-line packet capture. Can filter by UDP port (443 is common for QUIC). tshark (Wireshark's

CLI) can dissect QUIC packets and output structured data.

Packet replay limitations:

Traditional packet replay tools (tcpreplay) can replay captured QUIC traffic at the UDP level, but the replayed traffic will not be "live" QUIC — the connection IDs, encryption, and sequence numbers will not match any active connection. This is sufficient for testing pattern-based detection (size/timing patterns) but not for testing decryption-based analysis. For the latter, we need to generate live QUIC traffic.

10.4 Network Emulation

To test our system under various network conditions, we need network emulation:

tc (Traffic Control) / netem (Network Emulator) on Linux:

Can add delay, jitter, packet loss, bandwidth limits, and reordering on a Linux interface. Essential for testing how our classification performs under degraded network conditions (which alter traffic patterns).

Mininet:

Network emulation tool that creates virtual network topologies. Can run actual QUIC endpoints on virtual hosts with configurable link properties.

Mahimahi:

A set of network emulation tools specifically designed for web performance research. Can record and replay network conditions (link capacity traces). Particularly useful for testing adaptive video streaming behavior.

10.5 Recommended Setup for Our Project

A practical simulation environment for our project would look like:

[Diagram: A setup showing: QUIC Client (browser/curl/aioquic) → Linux machine running tc/netem for network emulation → QUIC Server (local HTTP/3 server or public server). A "Traffic Analyzer" component taps the traffic between client and server (via packet capture or inline). The Traffic Analyzer feeds into a "Classification Engine" and a "Policing Engine".]

For generating diverse traffic:

1. **Video streaming:** Play YouTube videos in Chrome (generates real QUIC to Google servers) or set up a local HTTP/3 server serving video segments
2. **Web browsing:** Automated browser scripts visiting various websites
3. **VoIP simulation:** Custom aioquic application sending periodic small packets
4. **File transfer:** curl HTTP/3 downloads of large files
5. **DNS over QUIC:** Use a DoQ client against a public DoQ server (like dns.adguard.com)

- 6. **Background UDP noise:** iperf UDP flows, DNS, NTP, gaming traffic — for testing QUIC vs. non-QUIC discrimination

11. Policy Walkthrough

This is where the "so what" of our project comes together. Once we have identified and classified QUIC flows, what policing actions do we take, and how?

11.1 Why Policy Control for QUIC Is Needed

Traditional QoS and policy infrastructure was built around inspectable traffic:

- **IP-level QoS (DiffServ):** Works by marking packets with DSCP values. Still works for QUIC since it operates at the IP level. However, the DSCP values are set by the endpoints (or possibly overwritten by the network), and QUIC endpoints typically set DSCP based on the application's requirements. The network cannot independently determine the "right" DSCP for a QUIC flow without classifying it first.
- **L7 DPI-based policies:** Completely broken for QUIC. An enterprise firewall that previously identified YouTube traffic by matching HTTP Host headers or URL patterns cannot do this for QUIC/HTTP/3 traffic without additional techniques. The immediate impulse for many enterprises has been to simply block QUIC (by blocking UDP port 443), forcing browsers to fall back to TCP/TLS. This is a stop-gap, not a solution.
- **Flow-based rate limiting:** Can still work for QUIC since we can identify flows by their UDP 5-tuple (or CID). But without classification, we are applying the same policy to all QUIC flows, which is not optimal — video streaming and VoIP have very different QoS requirements.

The value proposition of our project is clear: enable flow classification so that appropriate per-class policies can be applied to QUIC traffic.

11.2 Policy Goals

Before defining mechanisms, we need to be clear about what policies we want to enforce:

Bandwidth allocation: Ensure that video streaming does not consume all available bandwidth and starve other applications. Conversely, ensure that VoIP gets enough bandwidth for acceptable quality.

Prioritization: Give latency-sensitive traffic (VoIP, gaming) priority over latency-tolerant traffic (file downloads, software updates).

Rate limiting: Enforce per-user or per-application bandwidth caps (e.g., "no user may use more than 10 Mbps for video streaming").

Fairness: Ensure fair sharing among users/flows, preventing a single heavy user from degrading others' experience.

Abuse prevention: Detect and throttle anomalous QUIC usage (e.g., QUIC-based tunneling used to bypass network policies, DDoS using QUIC amplification).

11.3 Token Bucket Algorithm

The token bucket is the most commonly used rate-limiting mechanism and is directly applicable to QUIC policing.

Concept: Imagine a bucket that holds tokens. Tokens are added to the bucket at a constant rate (the *token rate*, r). The bucket has a maximum capacity (the *burst size*, b). When a packet arrives, it "consumes" tokens equal to its size. If enough tokens are available, the packet is conformant and is forwarded. If not, the packet is non-conformant and is either dropped, delayed, or marked for lower priority.

Parameters:

- **Token rate (r):** The sustained rate at which traffic is allowed. E.g., 5 Mbps. This is the long-term average rate the flow is policed to.
- **Burst size (b):** The maximum burst that can be accommodated above the sustained rate. This accounts for the inherently bursty nature of network traffic. E.g., 64 KB. A larger burst size allows more short-term burstiness but makes the rate limit less strict in the short term.

Behavior:

- When the bucket is full (after an idle period), a burst of up to b bytes can be sent at line rate.
- After the burst, the rate is limited to r until the bucket refills.
- If traffic arrives at a rate $\leq r$, the bucket stays full and all packets are conformant.
- If traffic arrives at a rate $> r$, the bucket drains. Once empty, non-conformant packets accumulate.

Token Bucket Applied to QUIC:

- Each classified QUIC flow (or group of flows) gets its own token bucket instance.
- Packet size for token consumption = UDP datagram size (since we are policing at the UDP level).
- Token rate is set based on the classified application type and the policy. E.g., video streaming flows might be policed to 8 Mbps, VoIP to 256 kbps, file downloads to 2 Mbps.
- Burst size should be tuned to the application's burst characteristics. Video streaming has large bursts (segment downloads), so a larger burst size is needed to avoid penalizing normal behavior. VoIP has minimal bursts, so a small burst size is fine.

11.4 Single Rate Three Color Marker (srTCM)

An extension of the token bucket that marks packets as green, yellow, or red based on conformance:

Two buckets:

- **Committed Bucket (CBS — Committed Burst Size):** Filled at the Committed Information Rate (CIR)
- **Excess Bucket (EBS — Excess Burst Size):** Holds overflow tokens from the Committed Bucket

Marking:

- If the packet fits within the Committed Bucket → **Green** (fully conformant, forward normally)
- If the Committed Bucket is insufficient but the Excess Bucket has tokens → **Yellow** (exceeding committed rate but within excess, forward with lower priority / best effort)
- If neither bucket has sufficient tokens → **Red** (non-conformant, drop or mark for very low priority)

This three-color scheme provides more granular handling than simple accept/drop. For QUIC, this allows us to handle normal operation (green), moderate bursts (yellow), and persistent overuse (red) differently.

11.5 Two Rate Three Color Marker (trTCM)

Similar to srTCM but with two independent rates:

- **CIR (Committed Information Rate) + CBS:** The guaranteed rate
- **PIR (Peak Information Rate) + PBS (Peak Burst Size):** The peak allowable rate

Packets are marked based on conformance to both rates. This is useful when we want to guarantee a base rate for each flow while allowing bursting up to a peak rate when the network has spare capacity.

For QUIC policing:

- CIR could be set based on the minimum acceptable quality (e.g., 2 Mbps for 720p video)
- PIR could be set based on the maximum policy allows (e.g., 10 Mbps for 4K)
- Traffic between CIR and PIR is yellow — allowed when the network is uncongested, dropped first when congestion occurs

11.6 EWMA-Based Rate Estimation

Exponentially Weighted Moving Average (EWMA) provides a smoothed estimate of the current rate of a flow, which is useful for making policing decisions.

Concept: Maintain a running rate estimate that is updated with each packet arrival:

$$\text{rate_estimate} = \alpha \times \text{instantaneous_rate} + (1 - \alpha) \times \text{previous_rate_estimate}$$

Where α is the smoothing factor ($0 < \alpha < 1$). A smaller α gives more weight to historical data (smoother but slower to react). A larger α gives more weight to recent data (responsive but noisier).

How to compute instantaneous rate: The simplest approach is to compute bytes/time since the last packet: $\text{instantaneous_rate} = \text{packet_size} / \text{inter_arrival_time}$. A more robust approach uses a sliding time window.

EWMA for QUIC Flow Monitoring:

- Track the EWMA rate for each QUIC flow.
- Compare against the policy threshold for the flow's classified type.
- If the EWMA rate exceeds the threshold consistently, trigger policing action.
- The EWMA naturally handles the bursty nature of QUIC traffic (especially video streaming's on-off pattern) better than instantaneous rate checks, because it smooths out the short-term bursts.

Choice of α : This is critical and application-dependent:

- For video streaming, a lower α (e.g., 0.1-0.3) is appropriate because the traffic is inherently bursty and we want to average over multiple burst cycles.
- For VoIP, a higher α (e.g., 0.5-0.8) is fine because the traffic is steady and sudden changes in rate may indicate an anomaly.
- For general-purpose policing, $\alpha = 0.25$ to 0.5 is a common starting point.

11.7 Leaky Bucket Algorithm

The leaky bucket is the dual of the token bucket. Instead of adding tokens, imagine a bucket with a hole at the bottom that leaks at a constant rate:

- Packets are added to the bucket when they arrive.
- The bucket "leaks" at a constant rate (the policed rate).
- If the bucket overflows (too many packets arrive too quickly), excess packets are dropped.
- The bucket depth determines the maximum burst size.

The leaky bucket enforces a stricter rate than the token bucket — it smooths traffic to a constant rate rather than allowing bursts above the sustained rate. For QUIC, pure leaky bucket may be too restrictive because QUIC's congestion control inherently produces bursty behavior (packet trains from the congestion window). A token bucket with appropriate burst size is generally more suitable.

11.8 Hierarchical Policing

In practice, we likely want multiple levels of policing:

Per-flow policing: Each QUIC flow (identified by 5-tuple or CID) has its own rate limit based on its classification.

Per-user/subscriber policing: Aggregate rate limit across all flows from/to a specific user (identified by IP address or subscriber ID). This prevents a user from opening many QUIC connections to circumvent per-flow limits.

Per-class policing: Aggregate rate limit for a traffic class (e.g., all video streaming, all VoIP) across all users. This ensures that a specific application class does not dominate the link.

Link-level policing: Overall rate limit on QUIC traffic vs. other traffic types. An operator might decide that QUIC traffic should not exceed 60% of link capacity.

[Diagram: Hierarchical tree showing Link Total at the top, branching into "QUIC Traffic" and "Other Traffic". QUIC Traffic branches into "Video", "Web", "VoIP", "Other". Each class branches into individual users, and each user has individual flows.]

11.9 Scheduling and Prioritization (Complementary to Policing)

Policing (dropping or marking non-conformant packets) is one side of QoS. The other side is scheduling — determining the order in which conformant packets are transmitted when the link is congested.

Strict Priority Queuing: VoIP packets always go first, then interactive traffic, then bulk transfers. Simple but can starve lower-priority traffic.

Weighted Fair Queuing (WFQ): Each class gets a weight determining its share of bandwidth. When all classes are active, bandwidth is divided according to weights. When a class is idle, its share is redistributed. This is more fair but more complex.

Deficit Round Robin (DRR): Similar to WFQ but with a simpler implementation. Each queue gets a "quantum" (number of bytes it can send per round) proportional to its weight.

For QUIC, the scheduling decision is made at the packet level. Since QUIC multiplexes streams within a single connection, we cannot prioritize individual streams (we cannot see which stream a packet belongs to). We can only prioritize entire connections/flows.

11.10 DSCP Marking Strategy

A practical approach that integrates with existing network QoS infrastructure:

1. **Classify the QUIC flow** (using the techniques described earlier)
2. **Set the DSCP field** in the IP header based on the classification:
 - VoIP → EF (Expedited Forwarding, DSCP 46)
 - Video streaming → AF41 (Assured Forwarding class 4, DSCP 34) or AF31
 - Web browsing → AF21 or Default (DSCP 0)
 - Bulk file transfer → AF11 or CS1 (scavenger)
3. **Existing network QoS infrastructure** (routers, switches) handles the prioritization based on DSCP values

This approach leverages the network's existing QoS capabilities and does not require the classification system to also perform scheduling and queuing. It just needs to mark packets appropriately.

Important consideration: QUIC endpoints may set their own DSCP values. RFC 9000 allows endpoints to set DSCP based on the application's requirements. If the endpoint is trusted (e.g., an enterprise-managed device), these DSCP values can be used directly. If the endpoint is untrusted (e.g., an arbitrary client on the Internet), the endpoint's DSCP values should be overwritten based on the network's independent classification — otherwise, any application could claim high priority.

11.11 Challenges and Edge Cases in QUIC Policing

Challenge: Connection migration defeats per-flow state.

When a flow migrates, the policing state (token bucket level, EWMA estimate, classification) is lost. The migrated flow starts fresh, potentially getting a "free burst" before being re-classified and policed.

Consideration: For typical deployments, migration is infrequent (mostly mobile users switching networks). The impact is manageable. For critical deployments, per-subscriber policing (aggregated by user identity rather than per-flow) is more robust.

Challenge: Encrypted content prevents content-based policies.

An enterprise that wants to block specific YouTube videos or restrict access to certain categories of websites cannot do so by inspecting QUIC traffic. They can only work at the hostname level (via SNI) or the flow level.

Consideration: This is a fundamental limitation of QUIC's design (intentional). For enterprises that need content-level control, the options are: (1) deploy a QUIC-terminating proxy that decrypts, inspects, and re-encrypts traffic; (2) block QUIC entirely and force TCP fallback where DPI works; or (3) use endpoint-based controls (agent on the device). Our project focuses on what can be done without terminating the encryption.

Challenge: QUIC version and implementation diversity.

Different QUIC versions and implementations may behave differently, affecting our classification accuracy. For example, Google's QUIC and Facebook's QUIC may produce different packet patterns

for similar application types due to different congestion control algorithms, pacing behavior, and implementation choices.

Consideration: Our classification system should be trained on diverse traffic from multiple implementations and should use features that are robust to implementation differences (like overall directionality and duration rather than precise packet sizes that depend on implementation-specific framing choices).

Challenge: Adversarial evasion.

If users or applications know about our classification system, they could try to evade it — by padding packets to alter size distributions, by adding artificial delays to disrupt timing patterns, or by tunneling one type of traffic inside another.

Consideration: For legitimate users (the vast majority), this is not a practical concern — browsers and applications are not going to add evasion logic to bypass QoS. For adversarial scenarios (VPN tunneling over QUIC to bypass restrictions), more sophisticated detection may be needed, but this is an edge case for our initial design.

Challenge: Multi-purpose connections.

A single QUIC connection may carry multiple types of traffic (e.g., a web page with embedded video, or a WebTransport session carrying both chat messages and media). Classifying the entire connection as one type is a simplification.

Consideration: Accept this limitation for the initial design. Classify based on the dominant traffic pattern. If the connection's pattern changes significantly mid-session (e.g., transitions from web browsing to video streaming), re-classify. This requires periodic re-evaluation of the classification decision rather than a one-time label.

Challenge: Reaction time vs. accuracy trade-off.

If we wait longer to collect more data, our classification is more accurate but the policing action is delayed (possibly by seconds or even tens of seconds). If we classify early (after just a few packets), we may misclassify and apply the wrong policy.

Consideration: A tiered approach: apply a default policy immediately (e.g., best-effort), make an initial classification after the first 10-20 packets, and refine the classification over the next few seconds. Each classification update adjusts the policing parameters. If a misclassification is detected (e.g., a flow initially classified as web browsing turns out to be video streaming after 10 seconds), the policing state is updated accordingly. The important thing is to handle transitions gracefully without causing packet loss during reclassification.

Challenge: Scalability.

Maintaining per-flow state (classification, token bucket, EWMA) for potentially millions of concurrent QUIC flows on a high-throughput link requires significant memory and processing.

Consideration: Use efficient data structures (hash tables for flow lookup). Consider aggregated policing for large populations (per-subscriber or per-class rather than per-flow). Expire idle flow state aggressively. Sampling for classification (not every packet needs to be analyzed — periodic sampling can maintain classification accuracy).

11.12 Putting It All Together — A Policing Architecture

The overall policing workflow for QUIC traffic looks like this:

[Diagram: Flowchart showing:

1. "Packet Arrives" →
2. "Is it QUIC?" (check fixed bit, version field, port heuristics) →
 - No → "Apply default UDP policy"
 - Yes →
3. "Is this a known flow?" (lookup by CID or 5-tuple) →
 - No → "Create flow entry, extract Initial packet info (SNI, ALPN), apply default QUIC policy"
→
 - Yes →
4. "Is flow classified?" →
 - No → "Update feature vector, enough data for classification?" → Yes → "Classify (ML model or rule-based), set policing parameters" / No → "Continue with default policy"
 - Yes →
5. "Apply policing (token bucket check)" →
 - Conformant → "Forward packet (possibly with DSCP mark)"
 - Non-conformant → "Drop / Mark / Deprioritize"
6. "Periodically re-evaluate classification (sliding window)"]

This architecture separates the classification concern (what type of traffic is this?) from the enforcement concern (what do we do about it?). The classification feeds into the policing parameters, but they operate independently.

12. Distinguishing QUIC from Generic UDP — Revisited

Since this is fundamental to the first step of our pipeline ("QUIC flow detector"), let us consolidate the identification methods with more detail than the previous week's document.

12.1 Identification Heuristics (Ordered by Reliability)

1. Version Field Match (Highest reliability):

If the first byte has bit 7 set (long header) and bytes 1-4 match a known QUIC version, it is almost certainly a QUIC packet. This works for handshake packets only. Maintain a table of known versions:

- 0x00000001 (v1), 0x6b3343cf (v2)
- Draft versions (0xff0000xx)
- Greased versions follow a specific pattern defined in RFC 9369

2. Fixed Bit Check:

Bit 6 of the first byte must be 1 for all QUIC v1 packets. This is a necessary but not sufficient condition (random UDP traffic has a 50% chance of having this bit set). Useful as a quick pre-filter.

3. Port Heuristic:

QUIC for HTTP/3 typically uses UDP port 443. DNS over QUIC uses port 853. However, QUIC can technically run on any port, and some implementations use non-standard ports. Port is a useful first-pass filter but should not be the sole identifier.

4. Initial Packet Structure Validation:

For packets that match the version and fixed bit checks, further validate:

- DCID length field should be ≤ 20
- SCID length field should be ≤ 20
- The total header length implied by the CID lengths should not exceed the packet size
- Initial packets should be ≥ 1200 bytes
- The Length field (for Initial, Handshake, 0-RTT) should be consistent with the remaining packet size

5. Handshake Pattern:

Observe the first few packets of a flow. If the pattern matches the expected QUIC handshake — specifically: Client → Server large packet (Initial, ≥ 1200 bytes), Server → Client large packet(s) (Initial+Handshake), Client → Server (Handshake completion) — it strongly suggests QUIC.

6. Post-Handshake Short Header Identification:

Once the handshake is observed, we know the CID length and can validate subsequent short header packets. Short header packets should have bit 7 = 0 and bit 6 = 1, and the first byte is followed by the DCID (of the previously observed length).

12.2 Common Non-QUIC UDP Traffic to Distinguish From

Protocol	Typical Ports	Distinguishing Features
DNS	53	Small packets (<512 bytes typically), well-known port, identifiable query/response structure
NTP	123	Fixed 48-byte packets, port 123
DTLS (WebRTC, VPN)	Various	Similar header structure concerns; DTLS has its own version fields and content types. First byte distinguishes: DTLS starts with content type (20-25), QUIC has the header form bit. DTLS records have a specific structure.
RTP/RTCP	Even/odd port pairs	RTP has version=2 in the first 2 bits. Can conflict with QUIC's header form bit pattern. Payload type field helps distinguish.
WireGuard	51820 (default)	Fixed port, specific message type bytes at the start
Gaming	Various	Application-specific protocols, often identifiable by port and packet patterns
STUN/TURN	3478	Magic cookie (0x2112A442) at bytes 4-7

The most challenging case is distinguishing QUIC from DTLS, because both are encrypted protocols over UDP with similar packet sizes and timing patterns. The key differentiator is the first byte structure and the version field (QUIC) vs. content type and protocol version (DTLS).

12.3 Edge Cases in QUIC Identification

QUIC on non-standard ports: Some deployments use non-443 ports. Our detector should be port-agnostic and rely on packet structure rather than port alone.

QUIC version greasing: Clients may send Initial packets with greased versions to test version negotiation. These might not match our version table. The greasing pattern (versions of the form 0x?a?a?a?) should be recognized.

Non-QUIC traffic on port 443: Other UDP protocols may use port 443 (e.g., DTLS for WebRTC). Port alone is not sufficient.

Fragmented UDP: If UDP fragmentation occurs (rare in practice since QUIC performs its own PMTUD), the IP fragments may not contain the QUIC header, making identification impossible for

non-first fragments. QUIC implementations generally avoid IP fragmentation by keeping packets within the path MTU.

Coalesced packets: Multiple QUIC packets in one UDP datagram. Our parser needs to handle this — after parsing the first QUIC packet (which has a Length field in long header format), check if there is remaining data in the UDP datagram for additional QUIC packets.

13. Bringing It All Together — Key Observations and Considerations for Our Project

13.1 The Information Hierarchy

Our classification capability follows a hierarchy from most to least informative:

1. **SNI + ALPN** (from Initial packet decryption): Tells us *who* the client is talking to and *what protocol* they are using. Most informative but requires Initial decryption.
2. **Statistical traffic patterns** (packet sizes, timing, directionality): Tells us *what kind of activity* is happening. Requires observation over time.
3. **Flow-level metadata** (duration, total bytes, packet count): Tells us the *scale* of the activity. Requires the flow to complete or reach steady state.
4. **IP and port information:** Tells us the *endpoints*. Least informative due to CDNs and shared hosting.

For robust classification, we should combine multiple levels of this hierarchy. SNI alone tells us the server but not the activity (a YouTube connection might be browsing or streaming). Statistical patterns alone might confuse different applications with similar behavior. Together, they are powerful.

13.2 The Time-Accuracy Trade-off

There is a fundamental tension between how quickly we classify and how accurately:

- **At t=0 (connection start):** We know nothing.
- **At t=1 RTT (after Initial exchange):** We can extract SNI and ALPN. This already narrows the classification significantly.
- **At t=1-2 seconds (first ~50 packets):** Initial statistical features are available. Early classification is possible with moderate accuracy.
- **At t=5-10 seconds:** Statistical features stabilize. Classification accuracy reaches its practical maximum for most traffic types.
- **At t=30+ seconds:** Long-lived flow identification is reliable. Video streaming vs. web browsing is clear.

Our policing system should use a graduated approach: apply coarse policies early (based on SNI/ALPN), refine with statistical classification as data accumulates, and adjust continuously.

13.3 What We Cannot Do (Fundamental Limitations)

It is important to be honest about what our system fundamentally cannot achieve:

- **Cannot determine specific content** (which YouTube video, which website page, what the user is typing). This is by design — QUIC's encryption is working as intended.
- **Cannot reliably link migrated connections** without endpoint cooperation.
- **Cannot perfectly classify every flow.** Some flows will be ambiguous (low confidence). The system needs a "default" policy for unclassifiable flows.
- **Cannot prevent encryption-based evasion** if an adversary specifically targets our classification system. But this is a niche concern.
- **Cannot inspect individual streams** within a multiplexed connection. Our classification and policing granularity is per-connection, not per-stream.

13.4 Ethical and Legal Considerations

Even though we are not decrypting application data, our traffic analysis capabilities raise considerations:

- **Traffic analysis can reveal user behavior** (e.g., inferring what someone is watching based on traffic patterns). We should be mindful of privacy implications.
- **Enterprise vs. ISP vs. government contexts** have different legal frameworks for traffic analysis and policing. What is appropriate for an enterprise managing its own network may not be appropriate for an ISP.
- **Transparency:** Users should ideally be aware that traffic classification and policing are being applied.

For our project, we are operating in a technical research context, so these are contextual considerations rather than blockers. But they are worth noting in our final report.

14. Summary

This document has covered the following areas in detail:

1. **QUIC stream types** — bidirectional/unidirectional, client/server-initiated, and how HTTP/3 maps onto them, including non-HTTP protocols over QUIC.
2. **Traffic pattern identification** — distinctive characteristics of video streaming, web browsing, VoIP, file transfer, and gaming in terms of packet sizes, timing, directionality, duration, and burst

patterns.

3. **Packet size distributions and burst analysis** — how different application types produce different statistical distributions, and what features to compute.
4. **QUIC header walkthrough** — byte-level analysis of long and short headers, what each field contains, and what is visible to a network observer.
5. **Extractable encrypted information** — what can be obtained from Initial packet decryption (SNI, ALPN, transport parameters), TLS fingerprinting, and the spin bit.
6. **Payload pattern analysis** — why encrypted payloads still leak information through size and timing, and how to use this for classification.
7. **Flow duration analysis** — measuring and using flow duration for classification, challenges with idle periods and connection migration.
8. **Upload/download patterns** — directional asymmetry as a classification signal, temporal evolution of directionality.
9. **Traffic simulation tools** — available QUIC implementations, traffic generators, capture tools, and network emulators for testing.
10. **Policy mechanisms** — token bucket, srTCM, trTCM, EWMA rate estimation, hierarchical policing, DSCP marking, and the challenges specific to QUIC.

This theoretical foundation should provide a solid basis for moving into the practical implementation phase. The next logical steps would involve setting up a test environment, capturing labeled traffic samples, and beginning to implement the classification and policing pipeline.

References:

- RFC 9000 — QUIC: A UDP-Based Multiplexed and Secure Transport
- RFC 9001 — Using TLS to Secure QUIC
- RFC 9002 — QUIC Loss Detection and Congestion Control
- RFC 8999 — Version-Independent Properties of QUIC
- RFC 9114 — HTTP/3
- RFC 9369 — QUIC Version 2
- RFC 9250 — DNS over Dedicated QUIC Connections
- RFC 2697 — A Single Rate Three Color Marker (srTCM)
- RFC 2698 — A Two Rate Three Color Marker (trTCM)
- RFC 8899 — Packetization Layer Path MTU Discovery for Datagram Transports